Стохастический градиентный спуск. Часть 2

В первой части мы познакомились с алгоритмом SGD и использованием инерции. Во второй части занятия мы начнем изучать способы изменения размера шага в алгоритме стохастического градиентного спуска. Кроме того, мы рассмотрим еще один градиентный метод — метод наискорейшего спуска.

При этом прежде всего рассмотрим теоретическое обоснование того, зачем (в большинстве случаев) рекомендуется уменьшать размер шага в процессе обучения алгоритма SGD.

Продолжим работать в том же блокноте

SGD и размер шага

В процессе full batch градиентного спуска значение градиента $\nabla f(\theta)$ всегда указывает «правильное» направление обновления весов в сторону θ^*. Это обусловлено тем, что каждый раз мы рассчитываем градиент на *всем* наборе данных, и у нас *одна* функция потерь $f(\theta)$.

При этом в стохастическом градиентном спуске, поскольку мы на каждом шаге рассчитываем градиент $\nabla f_i(\theta)$ на разных данных, речь идет о *нескольких* функциях потерь $f_i(\theta)$, и градиент не всегда указывает в сторону θ^*.

Простой пример

Рассмотрим задачу оптимизации с единственным параметром θ и набором данных, состоящим из трех наблюдений (x_i, y_i).

```
# зададим пространство параметра
theta = np.linspace(-30, 30, 100)

# и данные
X = np.array([0.8, 0.7, 0.6])
y = np.array([-6, 3, 7])
```

Возьмем квадратичную функцию потерь. Для трех наблюдений она будет иметь вид

$$f(\theta) = \frac{1}{3}\sum_{i=1}^{3}(\theta x_i - y_i)^2$$

При этом отдельные функции потерь, рассчитываемые на основе каждого из наблюдений,

будут заданы через

$$f_1(\theta) = (\theta x_1 - y_1)^2$$

$$f_2(\theta) = (\theta x_2 - y_2)^2$$

$$f_3(\theta) = (\theta x_3 - y_3)^2$$

```
# объявим функцию потерь для всего набора данных и
def f(theta, x, y):
  return np.sum((x * theta - y) ** 2) / len(x)

# функцию для каждой пары (x_i, y_i)
def f_i(theta, x_i, y_i):
  return (x_i * theta - y_i) ** 2
```

Найдем градиенты общей функции потерь и функций отдельных наблюдений.

$$\nabla f(\theta) = 2 \sum_i x_i (x_i \theta - y_i)$$

$$\nabla f_i(\theta) = 2 x_i (x_i \theta - y_i)$$

```
# объявим градиент f_i в точке theta
def grad_i(theta, x_i, y_i):
  return (2 * x_i) * (x_i * theta - y_i)
```

Тогда минимумы этих функций можно найти через $\nabla f(\theta) = 0$ и $\nabla f_i(\theta) = 0$. Получаем.

$$\theta^* = \frac{\sum_i x_i y_i}{\sum_i x_i^2}, \quad \theta_i^* = \frac{y_i}{x_i}$$

Найдем θ^* общей функции потерь и значение $f(\theta^*)$.

```
# найдем минимум общей функции потерь и
# значение функции в этой точке
theta_opt = (np.dot(X, y) / np.dot(X, X)).round(2)
theta_opt, f(theta_opt, X, y).round(2)
```

```
(1.01, 15.41)
```

```
# минимум и значение функции в минимуме можно найти и так
from scipy.optimize import minimize
# x0 = 5 - это начальное приближение,
# с которого оптимизатор начнет искать минимум
min_ = minimize(f, x0 = 5, args = (X, y))
min_.x.round(2), min_.fun.round(2)
```

```
(array([1.01]), 15.41)
```

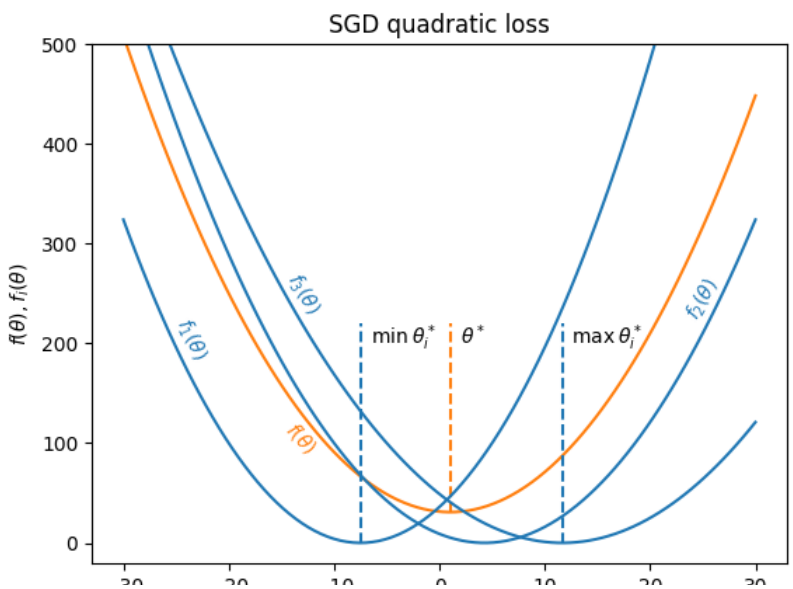
Также найдем значение каждой из трех θ_i^* и выведем минимальное $\min \theta_i^*$ и максимальное $\max \theta_i^*$ значения этого перечня (для чего это нужно, станет понятно чуть позже).

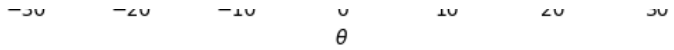
```
# найдем минимальные значения каждой из f_i
min_i_list = []
for x_i, y_i in zip(X, y):
  min_i_list.append((y_i/x_i).round(1))

# найдем минимальное и максимальное значения этих оптимумов
min_i_opt, max_i_opt = min(min_i_list), max(min_i_list)
min_i_opt, max_i_opt
```

```
(-7.5, 11.7)
```

Построим общую функцию потерь $f(\theta)$, функции $f_i(\theta)$, а также покажем общий минимум θ^* и минимальное и максимальное значения оптимумов $f_i(\theta)$.

```
# построим общую функцию потерь
plt.plot(theta, [f(theta_i, X, y) for theta_i in theta], c = 'C1')
# сделаем подпись и

plt.annotate(r'f(θ)', (-15, 90), rotation = -48, c = 'C1')
# покажем ее оптимум
plt.vlines(x = theta_opt, ymin = f(theta_opt, X, y), ymax = 220,
           colors = 'C1', linestyles = 'dashed')

plt.annotate(r'θ*', (2, 200))


# построим функции потерь отдельных наблюдений
for i, (x_i, y_i) in enumerate(zip(X, y), 1):
  plt.plot(theta, f_i(theta, x_i, y_i), c = 'C0')

# сделаем подпись и покажем минимум первой функции
# (он совпадает со значением минимального оптимума)

plt.annotate(r'f1(θ)', (-25.5, 185), rotation = -65, c = 'C0')
plt.vlines(x = min_i_opt, ymin = 0, ymax = 220, colors = 'C0', linestyles = 'dashed')

                    *

plt.annotate(r'min θi', (-6.5, 200))

# сделаем подпись ко второй функции

plt.annotate(r'f2(θ)', (23, 225), rotation = 60, c = 'C0')

# сделаем подпись и покажем минимум третьей функции
# (он совпадает со значением максимального оптимума)

plt.annotate(r'f3(θ)', (-15, 230), rotation = -60, c = 'C0')
plt.vlines(x = max_i_opt, ymin = 0, ymax = 220, colors = 'C0', linestyles = 'dashed')

                    *

plt.annotate(r'max θi', (12.5, 200))


plt.title('SGD quadratic loss')

plt.xlabel(r'θ')

plt.ylabel(r'f(θ),fi(θ)')
plt.ylim(-20, 500)
plt.show()
```





На графике хорошо видно, что при стохастическом градиентном спуске мы рассчитываем градиенты относительно трех функций потерь отдельных наблюдений. Покажем несколько возможных градиентов на графике.

Касательные к функции потерь

Возьмем три значения параметра θ' и построим касательные (градиенты) к функциям потерь $f_i(\theta)$ в этих точках. Пусть

- точки со значениями аргумента $\theta'_1 = -20$ и $\theta'_2 = -1$ лежат на функции $f_1(\theta)$; а
- точка $\theta'_3 = 25$ лежит на функции $f_2(\theta)$.

Найдем значения соответствующих функций $f_i(\theta')$ в этих точках.

```
# так как первые две точки лежат на функции f_1,
# мы используем данные первого наблюдения (X[0], y[0])
theta_prime_1 = -20
f_prime_1 = f_i(theta_prime_1, X[0], y[0])

theta_prime_2 = -1
f_prime_2 = f_i(theta_prime_2, X[0], y[0])

# третья точка лежит на функции f_2, и
# мы используем данные второго наблюдения (X[1], y[1])
theta_prime_3 = 25
f_prime_3 = f_i(theta_prime_3, X[1], y[1])
```

Зададим уравнение касательной.

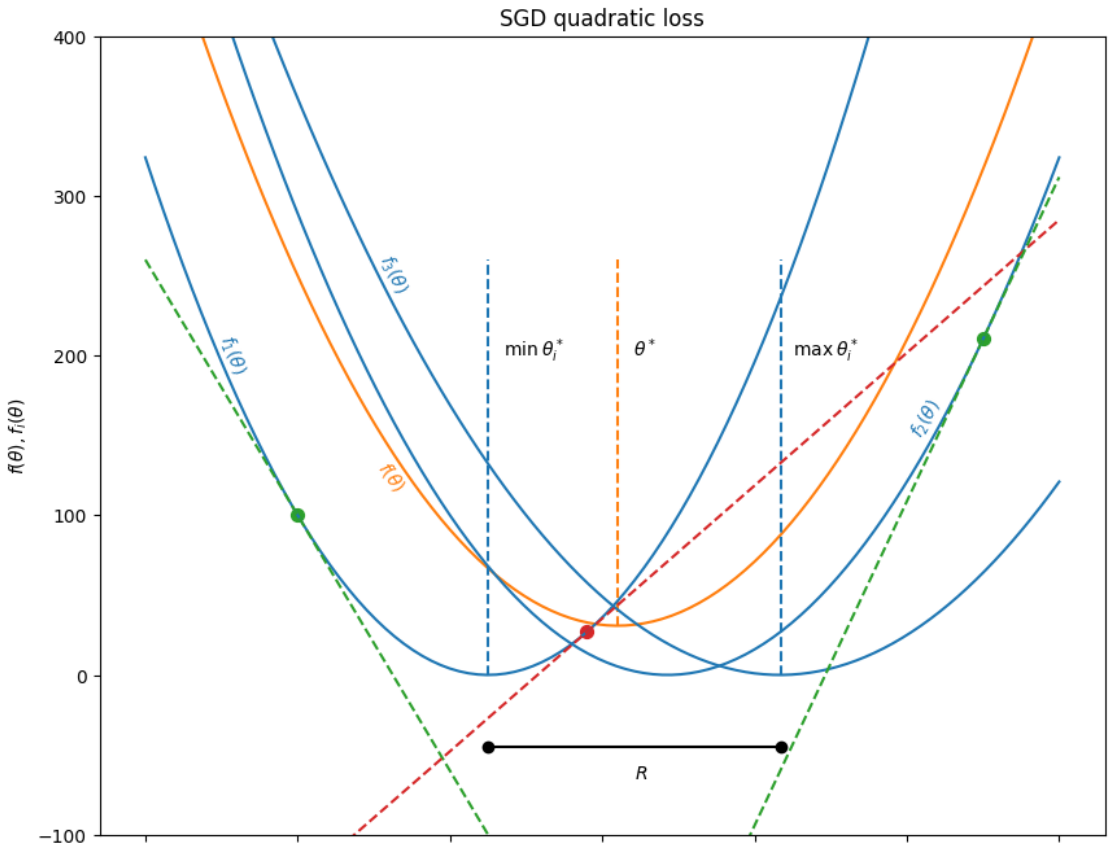
$$\text{tangent line} = \underbrace{\nabla f_i(\theta')}_{\text{slope}}(\theta - \theta') + f_i(\theta')$$

```
def line(theta_prime, f_prime, theta, x_i, y_i):
  return (grad_i(theta_prime, x_i, y_i) * (theta - theta_prime)) + f_prime
```

Вновь построим аналогичный график, только на этот раз покажем касательные в точках θ'.

```
plt.figure(figsize=(10,8))

# построим общую функцию потерь
plt.plot(theta, [f(theta_i, X, y) for theta_i in theta], c = 'C1')

plt.annotate(r'f(θ)', (-15, 115), rotation = -55, c = 'C1')

# покажем истинный оптимум
plt.vlines(x = theta_opt, ymin = f(theta_opt, X, y), ymax = 260,
           colors = 'C1', linestyles = 'dashed')

plt.annotate(r'θ*', (2, 200))

# построим функции потерь отдельных наблюдений f_i
for i, (x_i, y_i) in enumerate(zip(X, y), 1):
  plt.plot(theta, f_i(theta, x_i, y_i), c = 'C0')

# покажем минимальное и
plt.vlines(x = min_i_opt, ymin = 0, ymax = 260, colors = 'C0', linestyles = 'dashed')
```

```
                    *

plt.annotate(r'min θi', (-6.5, 200))
# максимальное значения оптимумов функций f_i
plt.vlines(x = max_i_opt, ymin = 0, ymax = 260, colors = 'C0', linestyles = 'dashed')
                    *

plt.annotate(r'max θi', (12.5, 200))

# сделаем подписи к f_i

plt.annotate(r'f1(θ)', (-25.5, 190), rotation = -70, c = 'C0')

plt.annotate(r'f2(θ)', (20, 150), rotation = 60, c = 'C0')

plt.annotate(r'f3(θ)', (-15, 240), rotation = -65, c = 'C0')

# построим касательные
plt.plot(theta, line(theta_prime_1, f_prime_1, theta, X[0], y[0]), '--', color = 'C2')
plt.scatter(theta_prime_1, f_prime_1, color = 'C2', s = 50)

plt.plot(theta, line(theta_prime_2, f_prime_2, theta, X[0], y[0]), '--', color = 'C3')
plt.scatter(theta_prime_2, f_prime_2, color = 'C3', s = 50)

plt.plot(theta, line(theta_prime_3, f_prime_3, theta, X[1], y[1]), '--', color = 'C2')
plt.scatter(theta_prime_3, f_prime_3, color = 'C2', s = 50)

# покажем интервал между минимальным и максимальным оптимумом f_i
plt.plot([min_i_opt, max_i_opt], [-45, -45], '-o', color='k')

plt.annotate(r'R ', ((min_i_opt + max_i_opt)/2, -65))

plt.title('SGD quadratic loss')

plt.xlabel(r'θ')

plt.ylabel(r'f(θ),fi(θ)')
plt.ylim(-100, 400)
plt.show()
```



Как мы сказали ранее, стохастический градиентный спуск является несмещенной оценкой full batch градиента.

$$\mathbb{E}[\nabla f_i(\theta)] = \nabla f(\theta)$$

Другими словами, в среднем градиенты отдельных функций должны привести нас к истинному минимуму θ^*. При этом, как видно на рисунке выше, в каждом отдельном случае градиенты $\nabla f_i(\theta)$ могут указывать:

- как в сторону θ^* и, таким образом, снижать *общую* ошибку (рассчитываем «зеленые» градиенты, однако ошибку измеряем по «оранжевой» функции);
- так и указывать в обратную от θ^* сторону и соответственно эту ошибку повышать (например, «красный» градиент).

Region of confusion

Более того, удаленные от θ^* «зеленые» градиенты всегда указывают «верное» направление, но по мере приближения к истинному минимуму, могут ошибаться.

> Можно также сказать, что «зеленые» градиенты $\nabla f_i(\theta)$ имеют *тот же знак*, что и градиенты общей функции потерь $\nabla f(\theta)$, а «красные» $\nabla f_i(\theta)$ могут совпадать с ними по знаку или не совпадать.

Интервал, в котором направление градиентов может изменяться, называют **region of confusion**, R. У него есть две особенности:

- границы этого интервала определены минимальным и максимальным значением оптимумов функций отдельных наблюдений $f_i(\theta)$ (именно для этого мы находили их выше);
- истинный минимум θ^* всегда находится внутри R (ведь за его пределами «зеленые» градиенты всегда совпадают по знаку с общим градиентом).

Более формально,

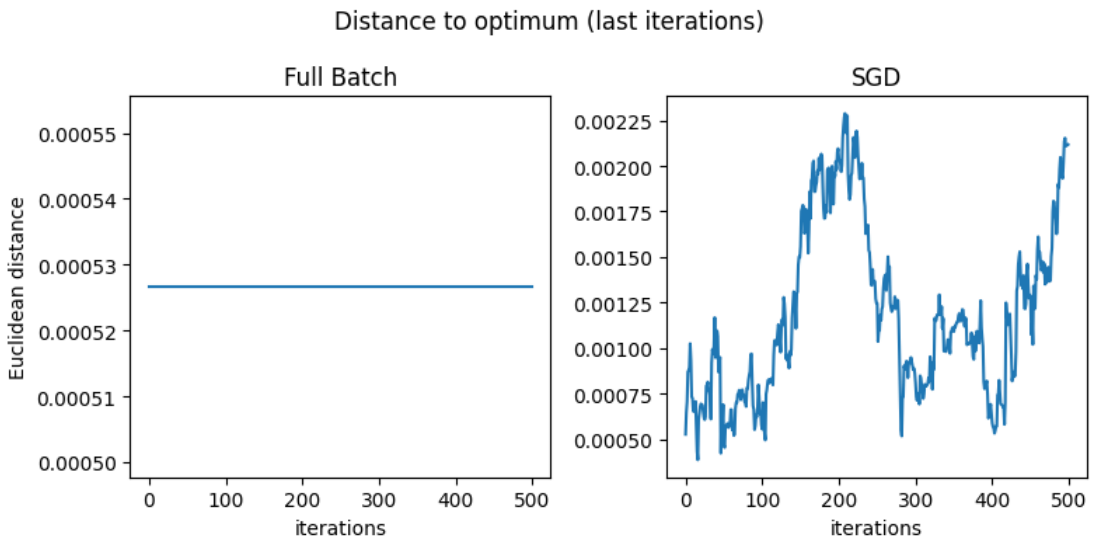
$$\theta^* \in \left[\min_i \theta_i^*,\ \max_i \theta_i^*\right] =: R$$

Это приводит к появлению двух фаз стохастического градиентного спуска:

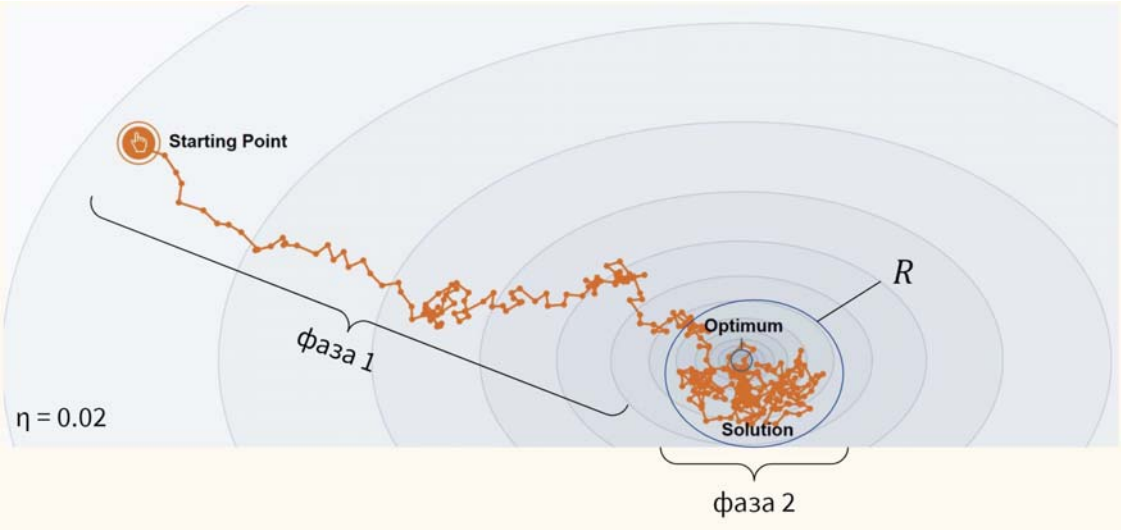
- уровень ошибки быстро снижается до определенного уровня («зеленые градиенты»); затем
- начинаются хаотичные незатухающие колебания («красные градиенты»).

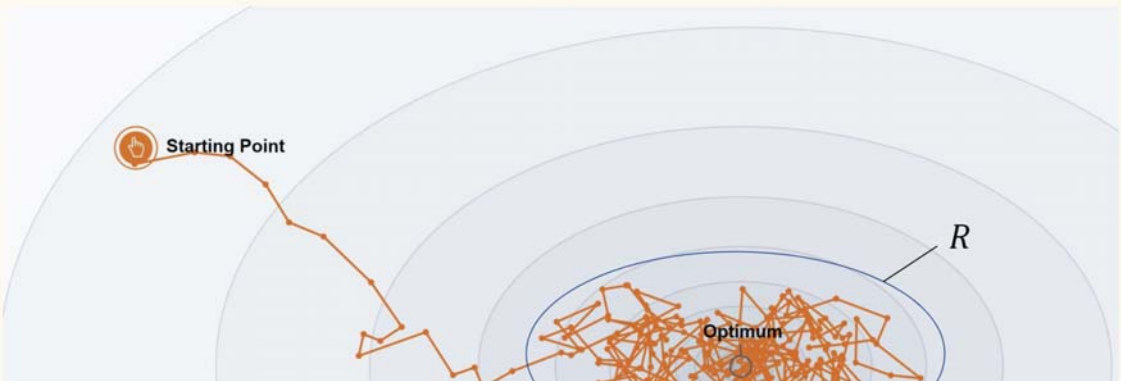
Приведем две иллюстрации. Вначале сравним расстояние до оптимума последних 500 итераций full batch и SGD алгоритмов, которые мы создали в первой части.

```python
fig = plt.figure(figsize = (8,4))
fig.suptitle('Distance to optimum (last iterations)')
ax1 = fig.add_subplot(121, title = 'Full Batch')
ax1.plot(dist_fb[-500:])
ax1.set_xlabel('iterations')
ax1.set_ylabel('Euclidean distance')
ax2 = fig.add_subplot(122, title = 'SGD')
ax2.plot(dist_sgd[-500:])
ax2.set_xlabel('iterations')
plt.tight_layout()
plt.show()
```



Теперь сравним процесс обучения алгоритма с функцией двух параметров $f(\theta_1, \theta_2)$ для размера шага $\eta = 0{,}02$ и $\eta = 0{,}06$.





Иллюстрации созданы здесь.

В этом примере мы также можем наблюдать две фазы SGD, однако речь идет уже не об интервале, а об окружности, диаметр которой описывает хаотические колебания второй фазы.

Заметим, что хотя SGD сохраняет незатухающие колебания, он подходит для решения практических задач, где нет необходимости достичь оптимума на обучающей выборке, но есть задача быстро снизить ошибку и показать хорошие результаты на тестовых данных.

Дисперсия градиента

Однако, в случае когда колебания *слишком* велики, алгоритм SGD может быть неэффективен. Опишем колебания с помощью **дисперсии** стохастического градиента относительно full-batch градиента

$$Var(\text{SGD}) = \frac{1}{n}\sum_{i=1}^{n} ||\nabla f_i(\theta) - \nabla f(\theta)||^2$$

Можно сказать, что

- за пределами region of confusion дисперсия равна нулю; а
- внутри этого интервала дисперсия больше нуля и увеличивается по мере приближения к θ^*.

При этом несложно заметить, что ширина интервала (диаметр окружности) пропорциональна размеру шага

$$R \propto \eta,$$

а дисперсия внутри R пропорциональна квадрату размера шага

$$Var(\text{SGD}) \propto \eta^2$$

> Как следствие, если мы хотим, чтобы стохастический градиентный спуск сходился к θ^*, нам необходимо постепенно уменьшать η.

Примечание. С точки зрения нотации изменяющийся размер шага мы будем обозначать как η_k. Другими словами, формула SGD будет иметь вид

$$\theta^{k+1} := \theta^k - \eta_k \cdot \nabla f_i(\theta^k), \quad k = 1, 2, 3, \ldots$$

Критерии сходимости

Формально сходимость стохастического градиентного спуска обеспечивается таким выбором последовательности η, для которой выполняются следующие условия

$$\sum_{k=1}^{\infty}(\eta_k) = \infty \quad \text{и} \quad \sum_{k=1}^{\infty}(\eta_k)^2 < \infty$$

Первое условие (1) описывает изменение размера шага по мере увеличения k. Нам важно, чтобы этот ряд расходился, поскольку так мы гарантированно «дойдем» до точки минимума. Согласно второму условию (2) дисперсия градиента должна стремиться к нулю.

Распространенным решением является последовательность $\eta_k = \frac{1}{k}$. И действительно при

$$\lim_{k\to\infty}\frac{1}{k} = 0$$

выполняется

$$\sum_{k=1}^{\infty}\left(\frac{1}{k}\right) = \infty, \qquad \sum_{k=1}^{\infty}\left(\frac{1}{k}\right)^2 = \frac{\pi^2}{6}$$

Расписания

На практике речь идет о применении различных расписаний (schedules), которые позволяют постепенно снижать размер шага (learning rate decay). Рассмотрим некоторые виды таких расписаний.

Inverse time decay

Размера шага можно уменьшать путем умножения на коэффициент, обратно пропорциональный эпохе.

$$\eta_k = \frac{\eta_0}{1 + r \cdot \text{epoch}},$$

где η_0 — начальный размер шага, а r — коэффициент затухания (decay rate).

```
def inv_time_decay(epoch, lr = 0.1, decay = 0.05):
  return lr / (1 + decay * epoch)
```

Заметим, что коэффициент затухания часто задают как отношение размера шага к количеству эпох.

$$r = \frac{\eta_0}{\text{epochs}}$$

Например,

```
lr, epochs = 0.1, 50

decay = lr / epochs
decay
```

```
0.002
```

Polynomial decay

Кроме этого, можно использовать полиномиальную функцию

$$\eta_k = \eta_0 \cdot \left(1 - \frac{\text{epoch}}{\text{epochs}}\right)^p,$$

где p указывает на степень полинома:

- чем меньше степень, тем медленнее происходит угасание размера шага;
- одновременно при $\eta = 1$ очевидным образом происходит линейное уменьшение шага.

Поскольку, следуя такому расписанию, на последнем шаге η будет равна нулю (а нам бы этого не хотелось), формулу модифицируют следующим образом

$$\eta_k = (\eta_{\text{start}} - \eta_{\text{end}}) \cdot \left(1 - \frac{\text{epoch}}{\text{epochs}}\right)^p + \eta_{\text{end}},$$

где η_{start} и η_{end} представляют собой гиперпараметры, определяющие начальный и конечный размер шага, и p — степень полинома.

```
# по умолчанию eta линейно уменьшается с 0,1 до 0,01 за 100 эпох
def polynomial_decay(epoch, epochs = 100, start_lr = 0.1, end_lr = 0.01, power = 1):
  return ((start_lr - end_lr) * (1 - epoch / epochs) ** power) + end_lr
```

Step-based decay

Помимо этого, размер шага можно уменьшать каждые несколько эпох (epochs drop) в определенное количество раз (drop rate).

$$\eta = \eta_0 \cdot \text{drop rate}^{\left\lfloor \frac{\text{epoch}}{\text{epochs drop}} \right\rfloor},$$

где $\lfloor\ \rfloor$ — функция пола для получения наибольшего целого числа, меньшего или равного заданному. Посмотрим, как это работает с помощью Питона. Пропишем функцию.

```
# по умолчанию eta уменьшается в два раза на каждой пятой эпохе
def step_decay(epoch, lr = 0.1, drop_rate = 0.5, epochs_drop = 5):
  return lr * (drop_rate ** np.floor(epoch / epochs_drop))
```

Теперь выведем размер шага для первых десяти эпох с параметрами по умолчанию (левая колонка ниже). Кроме того, выведем (правая колонка):

- отношение до функции пола $\frac{\text{epoch}}{\text{epochs drop}}$;
- результат после функции пола, то есть степень, в которую будет возводиться drop rate; а также
- коэффициент угасания drop rate на каждой эпохе.

```
drop_rate, epochs_drop = 0.5, 5

for epoch in range(15):
  print(step_decay(epoch), '\t', epoch / epochs_drop, np.floor(epoch / epochs_drop), \
        drop_rate ** np.floor(epoch / epochs_drop))
```

```
0.1      0.0 0.0 1.0
0.1      0.2 0.0 1.0
```

```
0.1      0.4 0.0 1.0
0.1      0.6 0.0 1.0
0.1      0.8 0.0 1.0
0.05     1.0 1.0 0.5
0.05     1.2 1.0 0.5
0.05     1.4 1.0 0.5
0.05     1.6 1.0 0.5
0.05     1.8 1.0 0.5
0.025    2.0 2.0 0.25
0.025    2.2 2.0 0.25
0.025    2.4 2.0 0.25
0.025    2.6 2.0 0.25
0.025    2.8 2.0 0.25
```

Например, на эпохах с 10 по 14 (если считать с нуля) η уменьшается в 4 раза и составляет $0{,}1 \cdot 0{,}25 = 0{,}025$.

Exponential decay

Размер шага можно также уменьшать по экспоненте

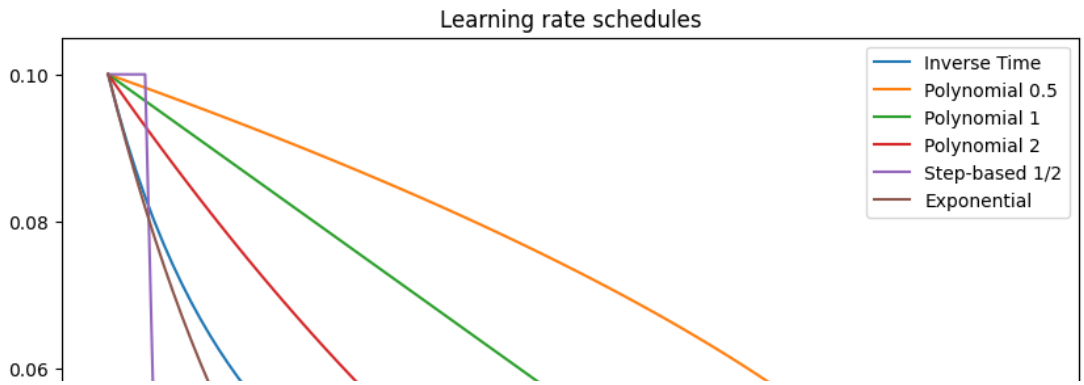
$$\eta_k = \eta_0 \cdot e^{-r \cdot \text{epoch}}$$

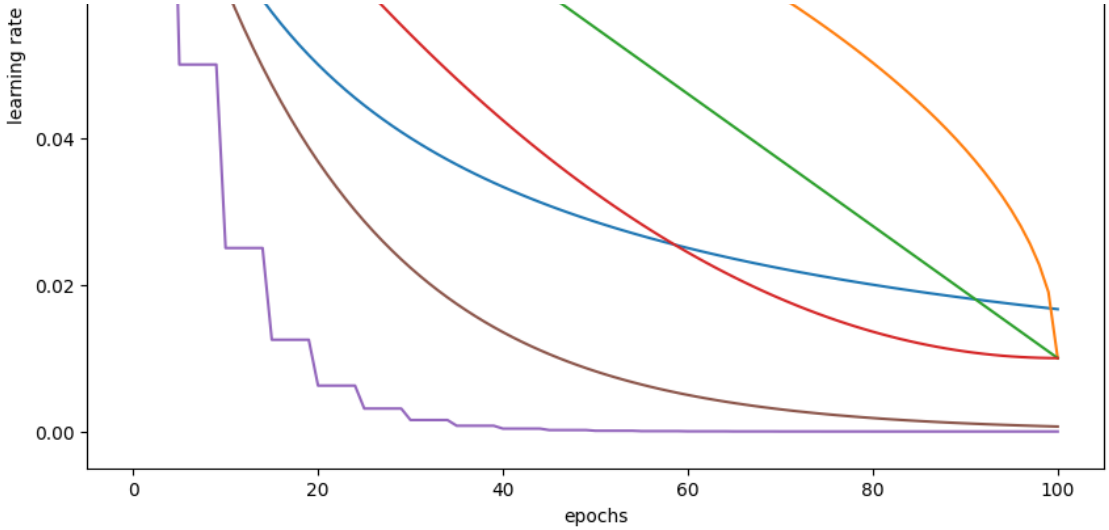
```
def exp_decay(epoch, lr = 0.1, decay = 0.05):
  return lr * np.exp(-decay * epoch)
```

Сравнение расписаний

Сравним расписания на графике.

```
epoch = np.arange(101)

plt.figure(figsize=(10,8))

plt.plot(epoch, inv_time_decay(epoch), label = 'Inverse Time')
plt.plot(epoch, polynomial_decay(epoch, power = 0.5), label = 'Polynomial 0.5')
plt.plot(epoch, polynomial_decay(epoch, power = 1), label = 'Polynomial 1')
plt.plot(epoch, polynomial_decay(epoch, power = 2), label = 'Polynomial 2')
plt.plot(epoch, step_decay(epoch), label = 'Step-based 1/2')
plt.plot(epoch, exp_decay(epoch), label = 'Exponential')
plt.legend()

plt.title('Learning rate schedules')
plt.xlabel('epochs')
plt.ylabel('learning rate')
plt.show()
```





Опять же обратим внимание, что в полиномиальной функции:

- наиболее быстро снижается полином с наиболее высокой степенью; кроме этого
- мы дошли ровно до конечной точки расписания с $\eta_{\text{end}} = 0{,}01$.

SGD с расписанием

Модифицируем алгоритм стохастического градиентного спуска.

```
def fit_sgd_schedule(x, y, t_opt, batch_size,
                     epochs = 50, lr = 0.1,
                     schedule = None,
                     decay = None,
                     epochs_drop = None,
                     end_lr = 0.01,
                     power = 1):

  x, y = x.copy(), y.copy()
  thetas = np.zeros(x.shape[1])

  # зададим eta_0
  initial_lr = lr

  dist_to_opt = []

  for epoch in range(epochs):
    batches = create_batches(x, y, batch_size)
    for x_b, y_b in batches:

      n = x_b.shape[0]

      dist_to_opt.append(np.linalg.norm(thetas - t_opt))

      grad = gradient(x_b, y_b, thetas, n)
      thetas = thetas - lr * grad

    if schedule == 'inv_time_decay':
      lr = initial_lr / (1 + decay * epoch)

    elif schedule == 'polynomial_decay':
      lr = ((initial_lr - end_lr) * (1 - epoch / epochs) ** power) + end_lr

    elif schedule == 'step_decay':
```

```
      lr = initial_lr * decay ** np.floor(epoch / epochs_drop)

    elif schedule == 'exp_decay':
      lr = initial_lr * np.exp(-decay * epoch)

  return dist_to_opt
```

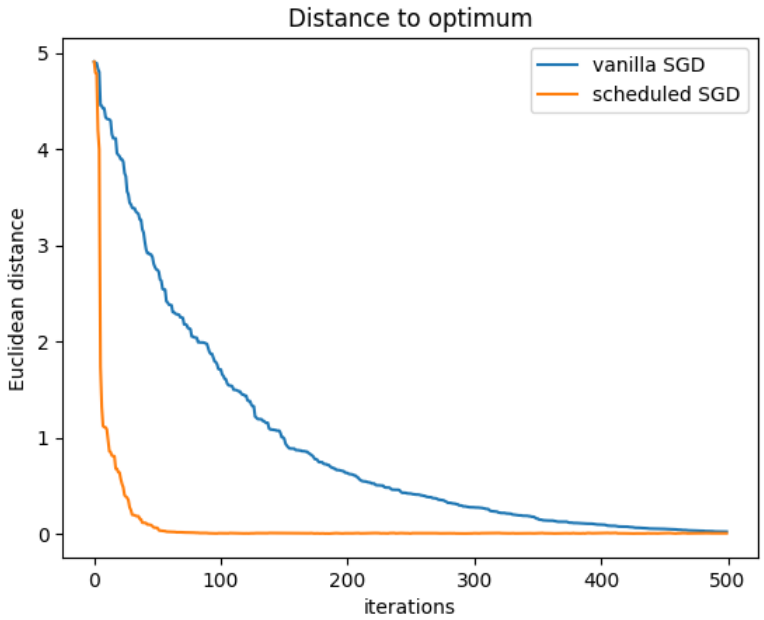
Применим алгоритм к синтетическим данным.

```
X, y = synth_reg_data(t_true)
```

```
dist_schedule = fit_sgd_schedule(X, y, t_true, batch_size = 1,
                                 epochs = 100, lr = 0.1,
                                 schedule = 'exp_decay',
                                 decay = 0.05)
```

Сравним обычный SGD (или как еще говорят vanilla SGD) с алгоритмом, который постепенно уменьшает размер шага по экспоненте.

```
plt.plot(dist_sgd[:500], label = 'vanilla SGD')
plt.plot(dist_schedule[:500], label = 'scheduled SGD')
plt.legend()

plt.title('Distance to optimum')
plt.xlabel('iterations')
plt.ylabel('Euclidean distance')
plt.show()
```



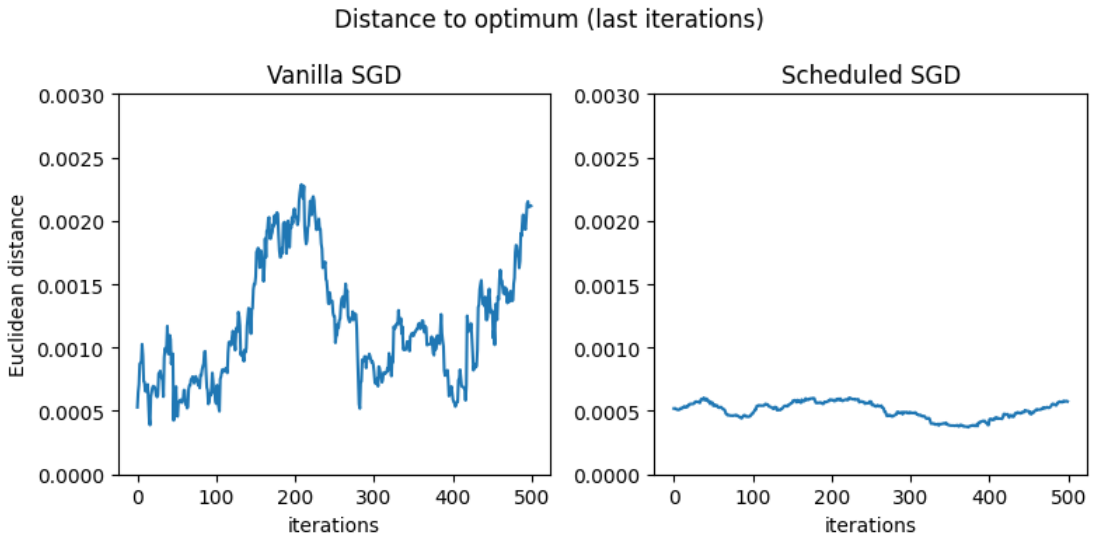
Разница на одних и тех же данных весьма существенная. Сделаем два замечания:

- во-первых, обратим внимание, что мы увеличили первоначальный размер шага, это оправданно, так как η будет довольно быстро снижаться;
- кроме того, заметим, что ничто не мешает объединять инерцию и уменьшение размера шага в одном алгоритме, выше мы не сделали этого ради упрощения кода.

Дополним изложение сравнением расстояния до оптимума на последних итерациях.

```
fig = plt.figure(figsize = (8,4))
fig.suptitle('Distance to optimum (last iterations)')
ax1 = fig.add_subplot(121, title = 'Vanilla SGD')
```

```
ax1.plot(dist_sgd[-500:])
ax1.set_xlabel('iterations')
ax1.set_ylabel('Euclidean distance')
ax1.set_ylim(0, 0.003)
ax2 = fig.add_subplot(122, title = 'Scheduled SGD')
ax2.plot(dist_schedule[-500:])
ax2.set_xlabel('iterations')
ax2.set_ylim(0, 0.003)
plt.tight_layout()
plt.show()
```



Мы видим, что на заключительных итерациях колебания стали существенно меньше.

Warmup и warm restarts

Размер шага можно не только уменьшать, но и также

- линейно увеличивать в начале обучения, и тогда говорят о «разогреве» коэффициента скорости обучения (learning rate linear warmup); или
- периодически «восстанавливать» до первоначального уровня после определенного количества эпох, а затем снова начинать уменьшение (warm restarts).

Во втором случае restart называется «теплым» (warm), поскольку при восстановлении размера шага веса не «сбрасываются» до нулевого или случайного значения, как это бывает при начале обучения и соответственно «холодном» старте (cold start).

Проиллюстрируем эти подходы.

Learing rate warmup

Приведем формулу линейного увеличения шага.

$$\eta_k = (\eta_{\text{target}} - \eta_{\text{init}}) \cdot \frac{\text{epoch}}{\text{warmup epochs}},$$

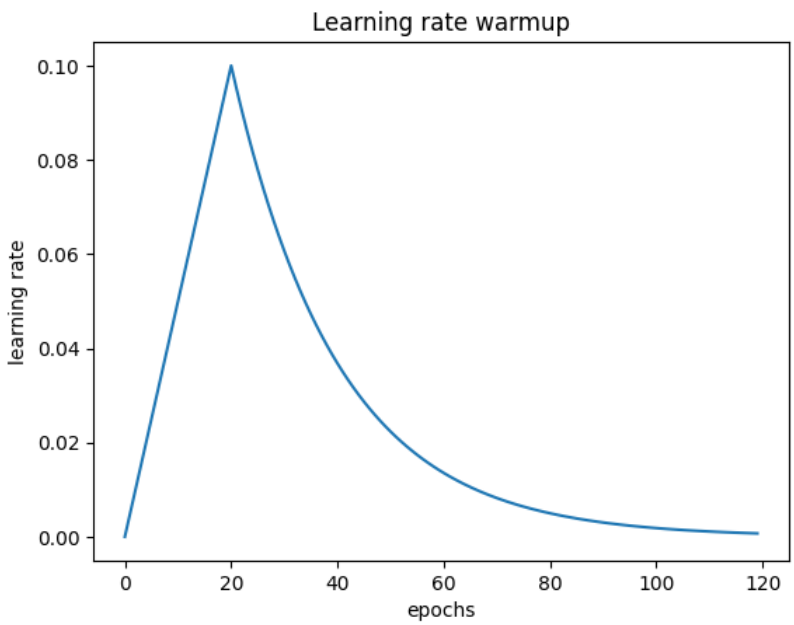
где η_{init} и η_{target} — размеры шага в начале и в конце фазы «разогрева».

```python
def warmup(epoch, init_lr, target_lr, warmup_epochs):
  return (target_lr - init_lr) * (epoch / warmup_epochs)
```

Покажем изменение η с warmup на обучении длинною 120 эпох.

```python
# создадим расписание с warmup
lr_schedule = []

# warmup будет длиться 20 эпох, убывание - 100
warmup_epochs, epochs = 20, 100

for epoch in range(warmup_epochs):
  # вначале eta будет возрастать от 0 до 0,1 в течение 20 эпох
  lr_schedule.append(warmup(epoch, 0, 0.1,
                            warmup_epochs = warmup_epochs))

for epoch in range(epochs):
  # затем eta будет убывать по экспоненте
  # с уровня 0,1 со скоростью 0,05
  lr_schedule.append(exp_decay(epoch, 0.1, 0.05))
```

```python
plt.plot(lr_schedule)
plt.title('Learning rate warmup')
plt.xlabel('epochs')
plt.ylabel('learning rate')
plt.show()
```



Warm restarts и cosine decay

Warm restarts можно обеспечить, если заставить η следовать косинусоиде. В этом случае говорят о cosine decay. Авторы метода предлагают следующее правило изменения размера шага

$$\eta_k = \eta_{\min} + \underbrace{\frac{1}{2}\left(\eta_{\max} - \eta_{\min}\right)}_{\text{amplitude}} \cdot \underbrace{\left(1 + \cos\left(\frac{\text{epoch}}{\text{epochs}} \cdot \pi\right)\right)}_{\text{decay}},$$

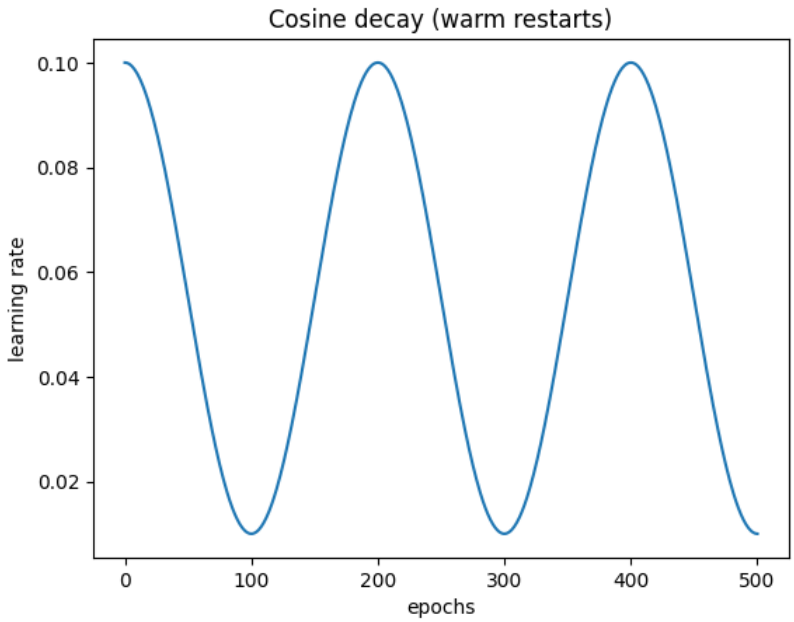
где $\eta_{\min}$ и $\eta_{\max}$ задают минимальный и максимальный размер шага. Заметим, что когда текущая epoch сравнивается с их общим количеством, то

$$\cos\pi = -1 \longrightarrow \eta_k = \eta_{\min}$$

```
epoch = np.arange(501)
```

```
def cosine_decay_1(epoch, lr_min = 0.01, lr_max = 0.1, epochs = 100):
  cos_decay = 1 + np.cos(np.pi * epoch / epochs)
  return lr_min + 0.5 * (lr_max - lr_min) * cos_decay
```

```
plt.plot(epoch, cosine_decay_1(epoch))
plt.title('Cosine decay (warm restarts)')
plt.xlabel('epochs')
plt.ylabel('learning rate')
plt.show()
```



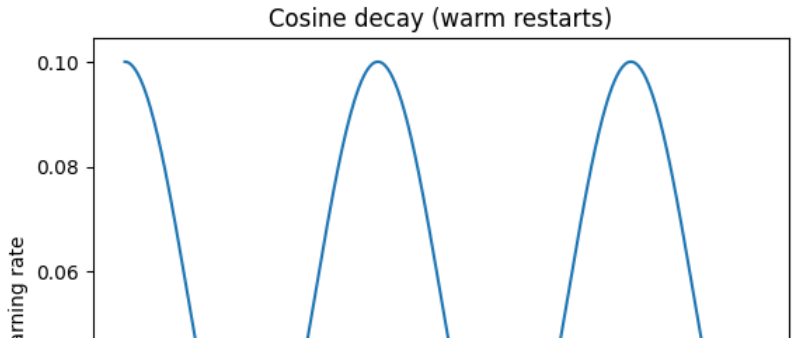
Одновременно, эту же формулу можно записать и так

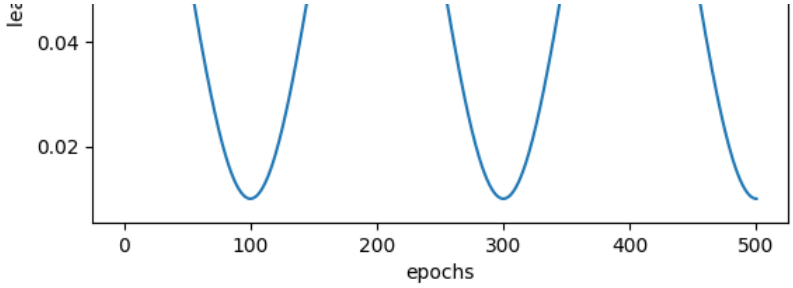
$$\eta_k = \eta_0 \cdot \left((1-\alpha) \cdot \left(1 + \cos\left(\frac{\text{epoch}}{\text{epochs}} \cdot \pi \right) \right) + \alpha \right)$$

В этом случае снижение размера шага происходит от η_0 до $(\alpha \cdot \eta_0)$.

```
def cosine_decay_2(epoch, initial_lr = 0.1, epochs = 100, alpha = 0.1):
  cosine_decay = 0.5 * (1 + np.cos(np.pi * epoch / epochs))
  decayed = (1 - alpha) * cosine_decay + alpha
  return initial_lr * decayed
```

```
plt.plot(epoch, cosine_decay_2(epoch))
plt.title('Cosine decay (warm restarts)')
plt.xlabel('epochs')
plt.ylabel('learning rate')
plt.show()
```





Как мы видим размер шага снижается с $\eta_0 = 0{,}1$ до $\alpha \cdot \eta_0 = 0{,}1 \cdot 0{,}1 = 0{,}01$.

Метод наискорейшего спуска

До сих пор мы так или иначе развивали идею стохастического градиентного спуска. Сделаем небольшое отступление и рассмотрим еще один метод оптимизации с изменяемым размером шага, который называется **методом наискорейшего спуска** (steepest descent method).

Базовый алгоритм

Вначале выполним подготовительную работу. Возьмем те же данные, целевую функцию и ее градиент.

```
t_true = np.array([3.2, 2, -2.7, 1.6])
X, y = synth_reg_data(t_true)
```

```
def objective(x, y, thetas):
  return np.sum((y - np.dot(x, thetas))) ** 2 / (2 * len(x))

def gradient(x, y, thetas):
  return np.dot(x.T, (np.dot(x, thetas) - y)) * (1 / len(x))
```

Создадим алгоритм full batch градиентного спуска. Другими словами, будем выполнять обновление весов в направлении антиградиента (направлении наискорейшего спуска).

При этом внесем в этот хорошо знакомый алгоритм одно изменение. Прекратим обучение не только по достижении максимального количества шагов (max_iter), но и если градиент будет ниже околонулевого порога (tol), то есть алгоритм практически приблизится к оптимуму.

Кроме того, запишем количество затраченных шагов, чтобы, если сработает второе условие, понять как быстро алгоритм достиг почти нулевого градиента.

```
# пропишем начальный размер шага, порог для градиента и
# максимальное количество шагов
eta, tol, max_iter = 0.01, 1e-8, 10000

# зададим начальные параметры
theta = np.zeros(X.shape[1])
# и значение градиента при таких параметрах
grad_theta = gradient(X, y, theta)

step_count = 0

# пока не сработает одно из условий
# (напомню any() выдает True, если хотя бы один из параметров
# ниже порогового значения)
```

```
while any(abs(grad_theta) > tol) and step_count < max_iter:

  # обновим веса и
  theta = theta - eta * grad_theta
  # рассчитаем градиент в новой точке
  grad_theta = gradient(X, y, theta)

  step_count += 1
```

```
theta, step_count
```

```
(array([ 3.19981975,  2.00017025, -2.70026591,  1.59961898]), 1960)
```

Как мы видим, алгоритм нашел оптимальные параметры и затратил на это 1960 шагов. Возникает вопрос, как решить эту же задачу за меньшее количество итераций.

Оптимальный размер шага

Одномерная оптимизация

Вполне очевидно, что уменьшить количество шагов можно за счет увеличения размера каждого отдельного шага. Для этого на каждом шаге «большого» алгоритма необходимо решить дополнительную задачу одномерной оптимизации относительно η.

$$\min_{\eta} f(\theta^k + \eta \nabla \theta^k),$$

где θ^k и $\nabla \theta^k$ — значения весов и градиента на текущем шаге k.

Условия Вольфе

Одновременно необходимо задать условия выбора оптимального параметра η. И действительно,

- если выбрать слишком большой шаг, алгоритм не сойдется к оптимуму;
- если размер шага будет недостаточным, алгоритм может не успеть дойти до минимума.

Таким образом, нам требуются два критерия:

1. уверенности, что значение функции уменьшится в новой точке θ_{k+1} (критерий достаточного убывания, sufficient decrease condition); а также
2. знания того, что размер шага не слишком мал (критерий кривизны, curvature condition).

Первый критерий называют **условием Армихо** (Armijo condition), а вместе эти критерии называют **условиями Вольфе** (Wolfe conditions). Более формально,

$$f(\theta_k + \eta_k \cdot p_k) \leq f(\theta_k) + c_1 \eta_k (\nabla f_k^T \cdot p_k),$$

$$\nabla f(\theta_k + \eta_k p_k)^T \cdot p_k \geq c_2 (\nabla f_k^T \cdot p_k)$$

где p_k (в данном случае) направление антиградиента, то есть $p_k = -\nabla f(\theta_k)$. При этом константы c_1 и c_2 выбираются таким образом, чтобы $0 < c_1 < c_2 < 1$. Подробное рассмотрение этих правил выходит за рамки сегодняшнего занятия.

Обратный линейный поиск

Подобрать оптимальный размер шага можно с помощью **обратного линейного поиска** (backtracking line search), при котором мы, как правило, начинаем с $\eta = 1$ и затем уменьшаем размер шага (например, в два раза) до тех пор, пока не будет выполнен критерий Армихо.

Интересно, что в этом случае достаточно указать только первый критерий, поскольку так как мы идем от большего размера шага к меньшему, то в любом случае выберем наибольшее возможное η и, таким образом, обеспечим выполнение и второго критерия Вольфе.

Такой поиск оптимального η называется приближенным линейным поиском (inexact line search), поскольку дает не точную, а достаточную оценку оптимального размера шага η на каждой итерации «большого» алгоритма.

Реализуем эту идею на Питоне и попробуем найти оптимальные параметры θ за меньшее количество шагов.

```
# пропишем алгоритм обратного линейного поиска
def line_search(eta, theta, grad_theta, x, y, c1 = 1e-4):

  # найдем значение целевой функции на шаге k
  objective_theta = objective(x, y, theta)

  # найдем значение целевой функции на шаге k+1
  objective_theta_new = objective(x, y, theta - eta * grad_theta)

  # найдем квадрат длины вектора (скалярное произведение)
  grad_sq = np.dot(grad_theta, grad_theta)

  # зададим условие, обратное условию Армихо
  not_armijo = objective_theta_new >= (objective_theta - c1 * eta * grad_sq)

  # пока истинно обратное условие,
  # будем уменьшать eta в два раза
  while not_armijo:
    eta *= 0.5

  return eta
```

```
# зададим порог значения градиента и
# максимальное количество итераций
tol, max_iter = 1e-8, 10000

# зададим начальные значения параметров и
theta = np.zeros(X.shape[1])
# начальный градиент
grad_theta = gradient(X, y, theta)

step_count = 0

# пока не сработает одно из условий
while any(abs(grad_theta) > tol) and step_count < max_iter:

  # будем искать оптимальное значение eta,
  # начиная с единицы
  eta_0 = 1
  eta = line_search(eta_0, theta, grad_theta, X, y)

  # обновлять веса и
  theta = theta - eta * grad_theta
  # обновлять градиент
  grad_theta = gradient(X, y, theta)
```

```

    step_count += 1
```

```
theta, step_count
```

```
(array([ 3.19981975,  2.00017025, -2.70026592,  1.59961898]), 8)
```

Метод наискорейшего спуска решил задачу за восемь шагов.

Еще немного терминологии

Классификация методов оптимизации

Итерационные методы оптимизации в зависимости от того, какая информация требуется для перехода от шага k к шагу $k+1$, можно разделить на:

- методы **нулевого порядка** (zero-order) или прямые методы, при которых требуется знать лишь значение целевой функции $f(\theta)$;
- методы **первого порядка** (first-order), которые используют первую производную $f'(\theta)$ (для одномерной функции) или градиент $\nabla f(\theta)$ (для многомерной);
- методы **второго порядка** (second-order), для которых необходимо знание второй производной $f''(\theta)$ или матрицы Гессе $\mathbf{H}f(\theta)$ (Гессиана функции, Hessian).

До сих пор мы подробно изучали методы первого порядка. В частности,

- метод градиентного спуска (в том числе SGD);
- метод координатного спуска; а также
- метод наискорейшего спуска.

Кроме этого скажем, что по характеру задачи методы делятся на:

- детерминированные (например, full batch градиентный спуск); и
- стохастические (SGD).

Оракул в оптимизации

При выполнении задачи численной оптимизации мы не имеем полного представления о целевой функции. Одновременно на каждой итерации метод может получать некоторую информацию о функции в точке θ_k. В частности, метод первого порядка может узнать значение функции и значение градиента.

Говорят, что в этом случае метод обращается к **оракулу** (oracle), абстрактной сущности, которая дает *локальную* информацию о функции в точке.

Например, можно говорить о классе стохастических оракулов первого порядка $\mathcal{O}$, в котором каждый оракул φ определяется через

$$\varphi(\theta, f) = (\hat{f}(\theta), \hat{g}(\theta)),$$

где $\hat{f}(\theta)$ и $\hat{g}(\theta)$ представляют собой случайные величины и являются несмещенной оценкой значения функции и значения градиента в точке θ соответственно

$$\mathbb{E}[\hat{f}(\theta)] = f(\theta), \quad \mathbb{E}[\hat{g}(\theta)] = g(\theta)$$

Промежуточный итог

Во второй части занятия мы рассмотрели теоретическое обоснование уменьшения размера шага в методе стохастического градиентного спуска, а также изучили как на практике коэффициент скорости обучения может изменяться с помощью расписания.

Кроме этого, мы рассмотрели еще один градиентный метод — метод наискорейшего спуска, ввели классификацию методов оптимизации и понятие оракула.

Список литературы

- Stochastic Gradient Descent with Warm Restarts, Loshchilov & Hutter, 2017
- On the Variance of the Adaptive Learning Rate and Beyond, Liu, 2021